

Trabajo Práctico N° 3: Bajas en Archivos y Actualización Maestro/Detalle, Reportes y Merge con Archivos no Ordenados.

PARTE I: Bajas en Archivos.

Ejercicio 1.

Modificar el Ejercicio 4 de la Práctica 1 (programa de gestión de empleados), agregándole una opción para realizar bajas copiando el último registro del archivo en la posición del registro a borrar y, luego, truncando el archivo en la posición del último registro de forma tal de evitar duplicados.

```
program TP3_E1;
{$codepage UTF8}
uses crt;
const
  apellido_salida='fin';
  edad_corte=70;
  dni_corte=0;
  opcion_salida=0;
type
  t_string10=string[10];
  t_registro_empleado=record
    numero: int16;
    apellido: t_string10;
    nombre: t_string10;
    edad: int8;
    dni: int32;
  end;
  t_archivo_empleados=file of t_registro_empleado;
function random_string(length: int8): t_string10;
var
  i: int8;
  string_aux: string;
begin
  string_aux:='';
  for i:= 1 to length do
    string_aux:=string_aux+chr(ord('A')+random(26));
  random_string:=string_aux;
end;
procedure leer_empleado(var registro_empleado: t_registro_empleado);
var
  i: int8;
begin
  i:=random(100);
  if (i=0) then
    registro_empleado.apellido:=apellido_salida
  else
    registro_empleado.apellido:=random_string(5+random(5));
  if (registro_empleado.apellido<>apellido_salida) then
    begin
      registro_empleado.numero:=1+random(1000);
      registro_empleado.nombre:=random_string(5+random(5));
      registro_empleado.edad:=18+random(high(int8)-18);
      if (i<=10) then
        registro_empleado.dni:=0
      else
        registro_empleado.dni:=10000000+random(40000001);
```

```
end;
end;
procedure cargar_archivo_empleados(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
begin
    rewrite(archivo_empleados);
    leer_empleado(registro_empleado);
    while (registro_empleado.apellido <> apellido_salida) do
    begin
        write(archivo_empleados, registro_empleado);
        leer_empleado(registro_empleado);
    end;
    close(archivo_empleados);
    textcolor(green); writeln('El archivo binario de empleados fue creado y cargado con éxito');
end;
procedure imprimir_registro_empleado(registro_empleado: t_registro_empleado);
begin
    textcolor(green); write('Número: '); textcolor(yellow); write(registro_empleado.numero);
    textcolor(green); write('; Apellido: '); textcolor(yellow);
write(registro_empleado.apellido);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_empleado.nombre);
    textcolor(green); write('; Edad: '); textcolor(yellow); write(registro_empleado.edad);
    textcolor(green); write('; DNI: '); textcolor(yellow); writeln(registro_empleado.dni);
end;
procedure imprimir1_archivo_empleados(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
    texto: t_string10;
begin
    texto:=random_string(5+random(5));
    reset(archivo_empleados);
    textcolor(green); write('Los datos de los empleados con nombre o apellido ');
textcolor(yellow); write(texto); textcolor(green); writeln(' son: ');
    while (not eof(archivo_empleados)) do
    begin
        read(archivo_empleados, registro_empleado);
        if ((registro_empleado.nombre=texto) or (registro_empleado.apellido=texto)) then
            imprimir_registro_empleado(registro_empleado);
        end;
        textcolor(green); write('Tamaño del archivo empleados: '); textcolor(yellow);
writeln(filesize(archivo_empleados));
        close(archivo_empleados);
    end;
end;
procedure imprimir2_archivo_empleados(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
begin
    reset(archivo_empleados);
    textcolor(green); writeln('Los empleados del archivo son: ');
    while (not eof(archivo_empleados)) do
    begin
        read(archivo_empleados, registro_empleado);
        imprimir_registro_empleado(registro_empleado);
    end;
    close(archivo_empleados);
end;
procedure imprimir3_archivo_empleados(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
begin
    reset(archivo_empleados);
    textcolor(green); write('Los empleados mayores a '); textcolor(yellow); write(edad_corte);
textcolor(green); writeln(' años son: ');
    while (not eof(archivo_empleados)) do
    begin
```

```
    read(archivo_empleados,registro_empleado);
    if (registro_empleado.edad>edad_corte) then
        imprimir_registro_empleado(registro_empleado);
    end;
    close(archivo_empleados);
end;
function control_unicidad(var archivo_empleados: t_archivo_empleados; numero: int16): boolean;
var
    registro_empleado: t_registro_empleado;
    ok: boolean;
begin
    ok:=false;
    while ((not eof(archivo_empleados)) and (ok=false)) do
        begin
            read(archivo_empleados,registro_empleado);
            if (registro_empleado.numero=numero) then
                ok:=true;
            end;
        end;
        control_unicidad:=ok;
    end;
end;
procedure agregar_empleado(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
    empleados: int16;
begin
    empleados:=0;
    reset(archivo_empleados);
    leer_empleado(registro_empleado);
    while (registro_empleado.apellido<>apellido_salida) do
        begin
            if (control_unicidad(archivo_empleados,registro_empleado.numero)=false) then
                begin
                    seek(archivo_empleados,filesize(archivo_empleados));
                    write(archivo_empleados,registro_empleado);
                    empleados:=empleados+1;
                end;
            leer_empleado(registro_empleado);
        end;
    end;
    close(archivo_empleados);
    textcolor(green); write('Se han agregado '); textcolor(yellow); write(empleados);
textcolor(green); writeln(' empleados al final del archivo');
end;
procedure modificar_edad_empleado(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
    numero: int16;
    ok: boolean;
begin
    numero:=1+random(1000);
    ok:=false;
    reset(archivo_empleados);
    while ((not eof(archivo_empleados)) and (ok=false)) do
        begin
            read(archivo_empleados,registro_empleado);
            if (registro_empleado.numero=numero) then
                begin
                    registro_empleado.edad:=18+random(high(int8)-18);
                    seek(archivo_empleados,filepos(archivo_empleados)-1);
                    write(archivo_empleados,registro_empleado);
                    ok:=true;
                end;
            end;
        end;
    end;
    close(archivo_empleados);
    if (ok=true) then
        begin
```

```

        textcolor(green); write('Se ha modificado la edad del empleado con número ');
textcolor(yellow); write(numero); textcolor(green); writeln(' en el archivo');
    end
    else
    begin
        textcolor(green); write('No se ha encontrado el empleado con número '); textcolor(yellow);
write(numero); textcolor(green); writeln(' en el archivo');
    end;
end;
procedure exportar_archivo_txt1(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
    archivo_txt: text;
begin
    reset(archivo_empleados);
    assign(archivo_txt,'E1_todos_empleados.txt'); rewrite(archivo_txt);
    while (not eof(archivo_empleados)) do
    begin
        read(archivo_empleados,registro_empleado);
        with registro_empleado do
            writeln(archivo_txt,'Número: ',numero,'; Apellido: ',apellido,'; Nombre: ',nombre,';
Edad: ',edad,'; DNI: ',dni);
        end;
        close(archivo_empleados);
        close(archivo_txt);
        textcolor(green); write('Se ha exportado el contenido del archivo al archivo de texto
llamado '); textcolor(yellow); writeln('"todos_empleados.txt"');
    end;
end;
procedure exportar_archivo_txt2(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
    archivo_txt: text;
begin
    reset(archivo_empleados);
    assign(archivo_txt,'E1_faltaDNIEmpleado.txt'); rewrite(archivo_txt);
    while (not eof(archivo_empleados)) do
    begin
        read(archivo_empleados,registro_empleado);
        if (registro_empleado.dni=dni_corte) then
            with registro_empleado do
                writeln(archivo_txt,'Número: ',numero,'; Apellido: ',apellido,'; Nombre: ',nombre,';
Edad: ',edad,'; DNI: ',dni);
            end;
        close(archivo_empleados);
        close(archivo_txt);
        textcolor(green); write('Se ha exportado a un archivo de texto llamado ');
textcolor(yellow); write('"faltaDNIEmpleado.txt"'); textcolor(green); writeln(' los empleados
que no tienen cargado el DNI (DNI en 00)');
    end;
end;
procedure eliminar_empleado(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado, registro_ultimo_empleado: t_registro_empleado;
    numero: int16;
begin
    numero:=1+random(1000);
    reset(archivo_empleados);
    seek(archivo_empleados,filesize(archivo_empleados)-1);
    read(archivo_empleados,registro_ultimo_empleado);
    seek(archivo_empleados,0);
    read(archivo_empleados,registro_empleado);
    while (not eof(archivo_empleados)) and (registro_empleado.numero<>numero) do
        read(archivo_empleados,registro_empleado);
    if (registro_empleado.numero=numero) then
    begin
        seek(archivo_empleados,filepos(archivo_empleados)-1);
        write(archivo_empleados,registro_ultimo_empleado);
    end;
end;

```

```

    seek(archivo_empleados, filesize(archivo_empleados)-1);
    truncate(archivo_empleados);
    textcolor(green); write('Se ha eliminado el empleado con número '); textcolor(yellow);
write(numero); textcolor(green); writeln(' en el archivo');
end
else
begin
    textcolor(green); write('No se ha encontrado el empleado con número '); textcolor(yellow);
write(numero); textcolor(green); writeln(' en el archivo');
end;
close(archivo_empleados);
end;
procedure leer_opcion(var opcion: int8);
begin
    textcolor(red); writeln('MENÚ DE OPCIONES');
    textcolor(yellow); write('OPCIÓN 1: '); textcolor(green); writeln('Crear un archivo de
registros no ordenados de empleados y completarlo con datos ingresados desde teclado');
    textcolor(yellow); write('OPCIÓN 2: '); textcolor(green); writeln('Listar en pantalla los
datos de empleados que tengan un nombre o apellido determinado');
    textcolor(yellow); write('OPCIÓN 3: '); textcolor(green); writeln('Listar en pantalla los
empleados de a uno por línea');
    textcolor(yellow); write('OPCIÓN 4: '); textcolor(green); writeln('Listar en pantalla los
empleados mayores a 70 años, próximos a jubilarse');
    textcolor(yellow); write('OPCIÓN 5: '); textcolor(green); writeln('Añadir uno o más
empleados al final del archivo con sus datos ingresados por teclado');
    textcolor(yellow); write('OPCIÓN 6: '); textcolor(green); writeln('Modificar la edad de un
empleado dado');
    textcolor(yellow); write('OPCIÓN 7: '); textcolor(green); writeln('Exportar el contenido del
archivo a un archivo de texto llamado "todos_empleados.txt"');
    textcolor(yellow); write('OPCIÓN 8: '); textcolor(green); writeln('Exportar a un archivo de
texto llamado "faltaDNIEmpleado.txt" los empleados que no tengan cargado el DNI (DNI en 00)');
    textcolor(yellow); write('OPCIÓN 9: '); textcolor(green); writeln('Eliminar un empleado del
archivo, copiando el último registro en la posición del registro a borrar y truncando el
archivo');
    textcolor(yellow); write('OPCIÓN 0: '); textcolor(green); writeln('Salir del menú de
opciones');
    textcolor(green); write('Introducir opción elegida: '); textcolor(yellow); readln(opcion);
    writeln();
end;
procedure menu_opciones(var archivo_empleados: t_archivo_empleados);
var
    opcion: int8;
begin
    leer_opcion(opcion);
    while (opcion<>opcion_salida) do
    begin
        case opcion of
            1: cargar_archivo_empleados(archivo_empleados);
            2: imprimir1_archivo_empleados(archivo_empleados);
            3: imprimir2_archivo_empleados(archivo_empleados);
            4: imprimir3_archivo_empleados(archivo_empleados);
            5: agregar_empleado(archivo_empleados);
            6: modificar_edad_empleado(archivo_empleados);
            7: exportar_archivo_txt1(archivo_empleados);
            8: exportar_archivo_txt2(archivo_empleados);
            9: eliminar_empleado(archivo_empleados);
            else
                textcolor(green); writeln('La opción ingresada no corresponde a ninguna de las
mostradas en el menú de opciones');
            end;
            writeln();
            leer_opcion(opcion);
        end;
    end;
end;
var
    archivo_empleados: t_archivo_empleados;

```

```
begin
  randomize;
  assign(archivo_empleados, 'E1_empleados');
  menu_opciones(archivo_empleados);
end.
```

Ejercicio 2.

Definir un programa que genere un archivo con registros de longitud fija conteniendo información de asistentes a un congreso a partir de la información obtenida por teclado. Se deberá almacenar la siguiente información: nro. de asistente, apellido y nombre, email, teléfono y DNI. Implementar un procedimiento que, a partir del archivo de datos generado, elimine, de forma lógica, todos los asistentes con nro. de asistente inferior a 1.000. Para ello, se podrá utilizar algún carácter especial situándolo delante de algún campo String a elección. Ejemplo: “@Saldaño”.

```

program TP3_E2;
{$codepage UTF8}
uses crt;
const
    numero_salida=0;
    numero_corte=1000;
type
    t_string20=string[20];
    t_registro_asistente=record
        numero: int16;
        apellido: t_string20;
        nombre: t_string20;
        email: t_string20;
        telefono: int32;
        dni: int32;
    end;
    t_archivo_asistentes=file of t_registro_asistente;
function random_string(length: int8): t_string20;
var
    i: int8;
    string_aux: string;
begin
    string_aux:='';
    for i:= 1 to length do
        string_aux:=string_aux+chr(ord('A')+random(26));
        random_string:=string_aux;
    end;
procedure leer_asistente(var registro_asistente: t_registro_asistente);
var
    i: int8;
begin
    i:=random(100);
    if (i=0) then
        registro_asistente.numero:=numero_salida
    else
        registro_asistente.numero:=1+random(2000);
        if (registro_asistente.numero<>numero_salida) then
            begin
                registro_asistente.apellido:=random_string(5+random(5));
                registro_asistente.nombre:=random_string(5+random(5));
                registro_asistente.email:=random_string(5+random(6))+ '@gmail.com';
                registro_asistente.telefono:=1000000000+random(1000000001);
                registro_asistente.dni:=10000000+random(40000001);
            end;
end;
procedure cargar_archivo_asistentes(var archivo_asistentes: t_archivo_asistentes);
var
    registro_asistente: t_registro_asistente;
begin
    rewrite(archivo_asistentes);
    leer_asistente(registro_asistente);
    while (registro_asistente.numero<>numero_salida) do

```

```

begin
    write(archivo_asistentes,registro_asistente);
    leer_asistente(registro_asistente);
end;
close(archivo_asistentes);
end;
procedure imprimir_registro_asistente(registro_asistente: t_registro_asistente);
begin
    textcolor(green); write('Número: '); textcolor(yellow); write(registro_asistente.numero);
    textcolor(green); write('; Apellido: '); textcolor(yellow);
write(registro_asistente.apellido);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_asistente.nombre);
    textcolor(green); write('; Email: '); textcolor(yellow); write(registro_asistente.email);
    textcolor(green); write('; Teléfono: '); textcolor(yellow);
write(registro_asistente.telefono);
    textcolor(green); write('; DNI: '); textcolor(yellow); writeln(registro_asistente.dni);
end;
procedure imprimir_archivo_asistentes(var archivo_asistentes: t_archivo_asistentes);
var
    registro_asistente: t_registro_asistente;
begin
    reset(archivo_asistentes);
    while (not eof(archivo_asistentes)) do
        begin
            read(archivo_asistentes,registro_asistente);
            imprimir_registro_asistente(registro_asistente);
        end;
        textcolor(green); write('Tamaño del archivo asistentes: '); textcolor(yellow);
writeln(filesize(archivo_asistentes));
        close(archivo_asistentes);
    end;
procedure eliminar_archivo_asistentes(var archivo_asistentes: t_archivo_asistentes);
var
    registro_asistente: t_registro_asistente;
begin
    reset(archivo_asistentes);
    while (not eof(archivo_asistentes)) do
        begin
            read(archivo_asistentes,registro_asistente);
            if (registro_asistente.numero<numero_corte) then
                begin
                    registro_asistente.apellido:='@'+registro_asistente.apellido;
                    seek(archivo_asistentes,filepos(archivo_asistentes)-1);
                    write(archivo_asistentes,registro_asistente);
                end;
            end;
        close(archivo_asistentes);
    end;
var
    archivo_asistentes: t_archivo_asistentes;
begin
    randomize;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO ASISTENTES:'); writeln();
    assign(archivo_asistentes,'E2_asistentes');
    cargar_archivo_asistentes(archivo_asistentes);
    imprimir_archivo_asistentes(archivo_asistentes);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO ASISTENTES ACTUALIZADO:'); writeln();
    eliminar_archivo_asistentes(archivo_asistentes);
    imprimir_archivo_asistentes(archivo_asistentes);
end.

```


Ejercicio 3.

Realizar un programa que genere un archivo de novelas filmadas durante el presente año. De cada novela, se registra: código, género, nombre, duración, director y precio. El programa debe presentar un menú con las siguientes opciones:

(a) *Crear el archivo y cargarlo a partir de datos ingresados por teclado. Se utiliza la técnica de lista invertida para recuperar espacio libre en el archivo. Para ello, durante la creación del archivo, en el primer registro del mismo, se debe almacenar la cabecera de la lista. Es decir, un registro ficticio, inicializando con el valor cero (0) el campo correspondiente al código de novela, el cual indica que no hay espacio libre dentro del archivo.*

(b) *Abrir el archivo existente y permitir su mantenimiento teniendo en cuenta el inciso (a), se utiliza lista invertida para recuperación de espacio. En particular, para el campo de “enlace” de la lista (utilizar el código de novela como enlace), se debe especificar los números de registro referenciados con signo negativo. Una vez abierto el archivo, brindar operaciones para:*

(i) *Dar de alta una novela leyendo la información desde teclado. Para esta operación, en caso de ser posible, deberá recuperarse el espacio libre. Es decir, si, en el campo correspondiente al código de novela del registro cabecera, hay un valor negativo, por ejemplo -5, se debe leer el registro en la posición 5, copiarlo en la posición 0 (actualizar la lista de espacio libre) y grabar el nuevo registro en la posición 5. Con el valor 0 (cero) en el registro cabecera se indica que no hay espacio libre.*

(ii) *Modificar los datos de una novela leyendo la información desde teclado. El código de novela no puede ser modificado.*

(iii) *Eliminar una novela cuyo código es ingresado por teclado. Por ejemplo, si se da de baja un registro en la posición 8, en el campo código de novela del registro cabecera, deberá figurar -8 y, en el registro en la posición 8, debe copiarse el antiguo registro cabecera.*

(c) *Listar, en un archivo de texto, todas las novelas, incluyendo las borradas, que representan la lista de espacio libre. El archivo debe llamarse “novelas.txt”.*

Nota: Tanto en la creación como en la apertura, el nombre del archivo debe ser proporcionado por el usuario.

```
program TP3_E3;
{$codepage UTF8}
uses crt;
const
    codigo_salida=-1;
    codigo_cabecera=0;
    opcion_salida=0;
type
    t_string10=string[10];
    t_registro_novela=record
        codigo: int16;
        genero: t_string10;
        nombre: t_string10;
```

```

    duracion: int16;
    director: t_string10;
    precio: real;
end;
t_archivo_novelas=file of t_registro_novela;
function random_string(length: int8): t_string10;
var
    i: int8;
    string_aux: string;
begin
    string_aux:='';
    for i:= 1 to length do
        string_aux:=string_aux+chr(ord('A')+random(26));
    random_string:=string_aux;
end;
procedure leer_cabecera(var registro_novela: t_registro_novela);
begin
    registro_novela.codigo:=codigo_cabecera;
    registro_novela.genero:='Cabecera';
    registro_novela.nombre:='Cabecera';
    registro_novela.duracion:=codigo_cabecera;
    registro_novela.director:='Cabecera';
    registro_novela.precio:=codigo_cabecera;
end;
procedure leer_novela(var registro_novela: t_registro_novela);
var
    i: int8;
begin
    i:=random(100);
    if (i=0) then
        registro_novela.codigo:=codigo_salida
    else
        registro_novela.codigo:=1+random(1000);
        if (registro_novela.codigo<>codigo_salida) then
            begin
                registro_novela.genero:=random_string(5+random(5));
                registro_novela.nombre:=random_string(5+random(5));
                registro_novela.duracion:=30+random(121);
                registro_novela.director:=random_string(5+random(5));
                registro_novela.precio:=1+random(100);
            end;
        end;
end;
procedure cargar_archivo_novelas(var archivo_novelas: t_archivo_novelas);
var
    registro_novela: t_registro_novela;
begin
    rewrite(archivo_novelas);
    leer_cabecera(registro_novela);
    while (registro_novela.codigo<>codigo_salida) do
        begin
            write(archivo_novelas,registro_novela);
            leer_novela(registro_novela);
        end;
    close(archivo_novelas);
    textcolor(green); writeln('El archivo binario de novelas fue creado y cargado con éxito');
end;
procedure imprimir_registro_novela(registro_novela: t_registro_novela);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_novela.codigo);
    textcolor(green); write('; Género: '); textcolor(yellow); write(registro_novela.genero);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_novela.nombre);
    textcolor(green); write('; Duración: '); textcolor(yellow); write(registro_novela.duracion);
    textcolor(green); write('; Director: '); textcolor(yellow); write(registro_novela.director);
    textcolor(green); write('; Precio: $'); textcolor(yellow);
    writeln(registro_novela.precio:0:2);
end;

```

```
procedure imprimir_archivo_novelas(var archivo_novelas: t_archivo_novelas);
var
    registro_novela: t_registro_novela;
begin
    reset(archivo_novelas);
    textcolor(green); writeln('Las novelas del archivo son: ');
    while (not eof(archivo_novelas)) do
    begin
        read(archivo_novelas, registro_novela);
        if (registro_novela.codigo > codigo_cabecera) then
            imprimir_registro_novela(registro_novela);
        end;
        textcolor(green); write('Tamaño del archivo novelas: '); textcolor(yellow);
        writeln(filesize(archivo_novelas));
        close(archivo_novelas);
    end;
procedure agregar_novela(var archivo_novelas: t_archivo_novelas);
var
    registro_novela, cabecera: t_registro_novela;
begin
    reset(archivo_novelas);
    read(archivo_novelas, cabecera);
    leer_novela(registro_novela);
    while (registro_novela.codigo = codigo_salida) do
        leer_novela(registro_novela);
    if (cabecera.codigo = codigo_cabecera) then
    begin
        seek(archivo_novelas, filesize(archivo_novelas));
        write(archivo_novelas, registro_novela);
    end
    else
    begin
        seek(archivo_novelas, cabecera.codigo * (-1));
        read(archivo_novelas, cabecera);
        seek(archivo_novelas, filepos(archivo_novelas) - 1);
        write(archivo_novelas, registro_novela);
        seek(archivo_novelas, 0);
        write(archivo_novelas, cabecera);
    end;
    close(archivo_novelas);
    textcolor(green); write('Se ha realizado el alta de la novela con código ');
textcolor(yellow); write(registro_novela.codigo); textcolor(green); writeln(' en el archivo');
end;
procedure modificar_novela(var archivo_novelas: t_archivo_novelas);
var
    registro_novela: t_registro_novela;
    codigo: int16;
    ok: boolean;
begin
    ok := false;
    codigo := 1 + random(1000);
    reset(archivo_novelas);
    while ((not eof(archivo_novelas)) and (ok = false)) do
    begin
        read(archivo_novelas, registro_novela);
        if (registro_novela.codigo = codigo) then
        begin
            ok := true;
            leer_novela(registro_novela);
            while (registro_novela.codigo = codigo_salida) do
                leer_novela(registro_novela);
            registro_novela.codigo := codigo;
            seek(archivo_novelas, filepos(archivo_novelas) - 1);
            write(archivo_novelas, registro_novela);
        end;
    end;
end;
```

```

close(archivo_novelas);
if (ok=true) then
begin
    textcolor(green); write('Se ha realizado la modificación de la novela con código ');
textcolor(yellow); write(codigo); textcolor(green); writeln(' en el archivo');
end
else
begin
    textcolor(green); write('No se ha encontrado la novela con código '); textcolor(yellow);
write(codigo); textcolor(green); writeln(' en el archivo');
end;
end;
procedure eliminar_novela(var archivo_novelas: t_archivo_novelas);
var
    registro_novela, cabecera: t_registro_novela;
    codigo: int16;
    ok: boolean;
begin
    ok:=false;
    codigo:=1+random(1000);
    reset(archivo_novelas);
    read(archivo_novelas,cabecera);
    while ((not eof(archivo_novelas)) and (ok=false)) do
    begin
        read(archivo_novelas,registro_novela);
        if (registro_novela.codigo=codigo) then
        begin
            ok:=true;
            seek(archivo_novelas,filepos(archivo_novelas)-1);
            write(archivo_novelas,cabecera);
            cabecera.codigo:=(filepos(archivo_novelas)-1)*(-1);
            seek(archivo_novelas,0);
            write(archivo_novelas,cabecera);
        end;
    end;
    close(archivo_novelas);
    if (ok=true) then
    begin
        textcolor(green); write('Se ha realizado la baja de la novela con código ');
textcolor(yellow); write(codigo); textcolor(green); writeln(' en el archivo');
    end
    else
    begin
        textcolor(green); write('No se ha encontrado la novela con código '); textcolor(yellow);
write(codigo); textcolor(green); writeln(' en el archivo');
    end;
end;
procedure exportar_archivo_txt(var archivo_novelas: t_archivo_novelas);
var
    registro_novela: t_registro_novela;
    archivo_txt: text;
begin
    reset(archivo_novelas);
    assign(archivo_txt,'E3_novelas.txt'); rewrite(archivo_txt);
    while (not eof(archivo_novelas)) do
    begin
        read(archivo_novelas,registro_novela);
        with registro_novela do
            writeln(archivo_txt,'Código: ',codigo,'; Género: ',genero,'; Nombre: ',nombre,';
Duración: ',duracion,'; Director: ',director,'; Precio: $',precio:0:2);
        end;
    end;
    close(archivo_novelas);
    close(archivo_txt);
    textcolor(green); write('Se han exportado todas las novelas, incluyendo las borradas, a un
archivo de texto llamado '); textcolor(yellow); writeln('"E3_novelas.txt"');
end;

```

```
procedure leer_opcion(var opcion: int8);
begin
    textcolor(red); writeln('MENÚ DE OPCIONES');
    textcolor(yellow); write('OPCIÓN 1: '); textcolor(green); writeln('Crear el archivo y
cargarlo a partir de datos ingresados por teclado');
    textcolor(yellow); write('OPCIÓN 2: '); textcolor(green); writeln('Listar en pantalla los
datos de las novelas del archivo, sin incluir las borradas');
    textcolor(yellow); write('OPCIÓN 3: '); textcolor(green); writeln('Dar de alta una novela
leyendo la información desde teclado');
    textcolor(yellow); write('OPCIÓN 4: '); textcolor(green); writeln('Modificar los datos de
una novela leyendo la información desde teclado');
    textcolor(yellow); write('OPCIÓN 5: '); textcolor(green); writeln('Eliminar una novela cuyo
código es ingresado por teclado');
    textcolor(yellow); write('OPCIÓN 6: '); textcolor(green); writeln('Exportar a un archivo de
texto llamado "novelas.txt" todas las novelas, incluyendo las borradas');
    textcolor(yellow); write('OPCIÓN 0: '); textcolor(green); writeln('Salir del menú de
opciones');
    textcolor(green); write('Introducir opción elegida: '); textcolor(yellow); readln(opcion);
    writeln();
end;
procedure menu_opciones(var archivo_novelas: t_archivo_novelas);
var
    opcion: int8;
begin
    leer_opcion(opcion);
    while (opcion<>opcion_salida) do
    begin
        case opcion of
            1: cargar_archivo_novelas(archivo_novelas);
            2: imprimir_archivo_novelas(archivo_novelas);
            3: agregar_novela(archivo_novelas);
            4: modificar_novela(archivo_novelas);
            5: eliminar_novela(archivo_novelas);
            6: exportar_archivo_txt(archivo_novelas);
        else
            writeln('La opción ingresada no corresponde a ninguna del menú de opciones');
        end;
        writeln();
        leer_opcion(opcion);
    end;
end;
var
    archivo_novelas: t_archivo_novelas;
begin
    randomize;
    assign(archivo_novelas, 'E3_novelas');
    menu_opciones(archivo_novelas);
end.
```

Ejercicio 4.

Dada la siguiente estructura:

```
type
  reg_flor=record
    nombre: String[45];
    codigo: integer;
  end;
  tArchFlores=file of reg_flor;
```

Las bajas se realizan apilando registros borrados y las altas reutilizando registros borrados. El registro 0 se usa como cabecera de la pila de registros borrados: el número 0 en el campo código implica que no hay registros borrados y -N indica que el próximo registro a reutilizar es el N, siendo éste un número relativo de registro válido.

(a) Implementar el siguiente módulo:

{Abre el archivo y agrega una flor recibida como parámetro, manteniendo la política descrita anteriormente}

procedure agregarFlor(var a: tArchFlores; nombre: string; codigo: integer);

(b) Listar el contenido del archivo omitiendo las flores eliminadas. Modificar lo que se considere necesario para obtener el listado.

```
program TP3_E4;
{$codepage UTF8}
uses crt;
const
  codigo_salida=-1;
  codigo_cabecera=0;
type
  t_string45=string[45];
  t_registro_flor=record
    nombre: t_string45;
    codigo: int16;
  end;
  t_archivo_flores=file of t_registro_flor;
function random_string(length: int8): t_string45;
var
  i: int8;
  string_aux: string;
begin
  string_aux:='';
  for i:= 1 to length do
    string_aux:=string_aux+chr(ord('A')+random(26));
  random_string:=string_aux;
end;
procedure leer_flor(var registro_flor: t_registro_flor);
var
  i: int8;
begin
  i:=random(100);
  if (i=0) then
    registro_flor.codigo:=codigo_salida
  else
    registro_flor.codigo:=1+random(1000);
```

```

    if (registro_flor.codigo<>codigo_salida) then
        registro_flor.nombre:=random_string(5+random(6));
    end;
procedure cargar_archivo_flores(var archivo_flores: t_archivo_flores);
var
    registro_flor: t_registro_flor;
begin
    rewrite(archivo_flores);
    registro_flor.codigo:=codigo_cabecera;
    registro_flor.nombre:='Cabecera';
    while (registro_flor.codigo<>codigo_salida) do
        begin
            write(archivo_flores,registro_flor);
            leer_flor(registro_flor);
        end;
    close(archivo_flores);
end;
procedure imprimir_registro_flor(registro_flor: t_registro_flor);
begin
    textcolor(green); write('Nombre: '); textcolor(yellow); write(registro_flor.nombre);
    textcolor(green); write('; Código: '); textcolor(yellow); writeln(registro_flor.codigo);
end;
procedure imprimir_archivo_flores(var archivo_flores: t_archivo_flores);
var
    registro_flor: t_registro_flor;
begin
    reset(archivo_flores);
    while (not eof(archivo_flores)) do
        begin
            read(archivo_flores,registro_flor);
            if (registro_flor.codigo>codigo_cabecera) then
                imprimir_registro_flor(registro_flor);
            end;
            textcolor(green); write('Tamaño del archivo flores: '); textcolor(yellow);
            writeln(filesize(archivo_flores));
            close(archivo_flores);
        end;
end;
procedure agregar_flor(var archivo_flores: t_archivo_flores; nombre: t_string45; codigo:
int16);
var
    registro_flor, cabecera: t_registro_flor;
begin
    reset(archivo_flores);
    read(archivo_flores,cabecera);
    registro_flor.nombre:=nombre;
    registro_flor.codigo:=codigo;
    if (cabecera.codigo=codigo_cabecera) then
        begin
            seek(archivo_flores,filesize(archivo_flores));
            write(archivo_flores,registro_flor);
        end
    else
        begin
            seek(archivo_flores,cabecera.codigo*(-1));
            read(archivo_flores,cabecera);
            seek(archivo_flores,filepos(archivo_flores)-1);
            write(archivo_flores,registro_flor);
            seek(archivo_flores,0);
            write(archivo_flores,cabecera);
        end;
    close(archivo_flores);
    textcolor(green); write('Se ha realizado el alta de la flor '); textcolor(yellow);
    write(nombre); textcolor(green); write(' con código '); textcolor(yellow); writeln(codigo);
    writeln();
end;
var

```

```
archivo_flores: t_archivo_flores;
codigo: int16;
nombre: t_string45;
begin
  randomize;
  writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO FLORES:'); writeln();
  assign(archivo_flores, 'E4_flores');
  cargar_archivo_flores(archivo_flores);
  imprimir_archivo_flores(archivo_flores);
  writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO FLORES AL AGREGAR FLOR:'); writeln();
  codigo:=1+random(1000);
  nombre:=random_string(5+random(6));
  agregar_flor(archivo_flores,nombre,codigo);
  imprimir_archivo_flores(archivo_flores);
end.
```


Ejercicio 5.

Dada la estructura planteada en el ejercicio anterior, implementar el siguiente módulo:

{Abre el archivo y elimina la flor recibida como parámetro, manteniendo la política descrita anteriormente}

procedure eliminarFlor(var a: tArchFlores; flor: reg_flor);

```

program TP3_E5;
{$codepage UTF8}
uses crt;
const
  codigo_salida=-1;
  codigo_cabecera=0;
type
  t_string45=string[45];
  t_registro_flor=record
    nombre: t_string45;
    codigo: int16;
  end;
  t_archivo_flores=file of t_registro_flor;
function random_string(length: int8): t_string45;
var
  i: int8;
  string_aux: string;
begin
  string_aux:='';
  for i:= 1 to length do
    string_aux:=string_aux+chr(ord('A')+random(26));
    random_string:=string_aux;
  end;
procedure leer_flor(var registro_flor: t_registro_flor);
var
  i: int8;
begin
  i:=random(100);
  if (i=0) then
    registro_flor.codigo:=codigo_salida
  else
    registro_flor.codigo:=1+random(100);
    if (registro_flor.codigo<>codigo_salida) then
      registro_flor.nombre:=random_string(5+random(6));
  end;
procedure cargar_archivo_flores(var archivo_flores: t_archivo_flores);
var
  registro_flor: t_registro_flor;
begin
  rewrite(archivo_flores);
  registro_flor.codigo:=codigo_cabecera;
  registro_flor.nombre:='Cabecera';
  while (registro_flor.codigo<>codigo_salida) do
    begin
      write(archivo_flores,registro_flor);
      leer_flor(registro_flor);
    end;
  close(archivo_flores);
end;
procedure imprimir_registro_flor(registro_flor: t_registro_flor);
begin
  textcolor(green); write('Nombre: '); textcolor(yellow); write(registro_flor.nombre);
  textcolor(green); write('; Código: '); textcolor(yellow); writeln(registro_flor.codigo);
end;

```

```
procedure imprimir_archivo_flores(var archivo_flores: t_archivo_flores);
var
    registro_flor: t_registro_flor;
begin
    reset(archivo_flores);
    while (not eof(archivo_flores)) do
        begin
            read(archivo_flores,registro_flor);
            if (registro_flor.codigo>codigo_cabecera) then
                imprimir_registro_flor(registro_flor);
            end;
            textcolor(green); write('Tamaño del archivo flores: '); textcolor(yellow);
            writeln(filesize(archivo_flores));
            close(archivo_flores);
        end;
    procedure agregar_flor(var archivo_flores: t_archivo_flores; nombre: t_string45; codigo:
        int16);
    var
        registro_flor, cabecera: t_registro_flor;
    begin
        reset(archivo_flores);
        read(archivo_flores,cabecera);
        registro_flor.nombre:=nombre;
        registro_flor.codigo:=codigo;
        if (cabecera.codigo=codigo_cabecera) then
            begin
                seek(archivo_flores,filesize(archivo_flores));
                write(archivo_flores,registro_flor);
            end
        else
            begin
                seek(archivo_flores,cabecera.codigo*(-1));
                read(archivo_flores,cabecera);
                seek(archivo_flores,filepos(archivo_flores)-1);
                write(archivo_flores,registro_flor);
                seek(archivo_flores,0);
                write(archivo_flores,cabecera);
            end;
        close(archivo_flores);
        textcolor(green); write('Se ha realizado el alta de la flor '); textcolor(yellow);
        write(nombre); textcolor(green); write(' con código '); textcolor(yellow); writeln(codigo);
        writeln();
    end;
    procedure eliminar_flor(var archivo_flores: t_archivo_flores; flor: t_registro_flor);
    var
        registro_flor, cabecera: t_registro_flor;
        ok: boolean;
    begin
        ok:=false;
        reset(archivo_flores);
        read(archivo_flores,cabecera);
        while ((not eof(archivo_flores)) and (ok=false)) do
            begin
                read(archivo_flores,registro_flor);
                if (registro_flor.codigo=flor.codigo) then
                    begin
                        ok:=true;
                        seek(archivo_flores,filepos(archivo_flores)-1);
                        write(archivo_flores,cabecera);
                        cabecera.codigo:=(filepos(archivo_flores)-1)*(-1);
                        seek(archivo_flores,0);
                        write(archivo_flores,cabecera);
                    end;
                end;
            end;
        close(archivo_flores);
        if (ok=true) then
```

```
begin
    textcolor(green); write('Se ha realizado la baja de la flor '); textcolor(yellow);
write(flor.nombre); textcolor(green); write(' con código '); textcolor(yellow);
writeln(flor.codigo);
end
else
begin
    textcolor(green); write('No se ha encontrado la flor '); textcolor(yellow);
write(flor.nombre); textcolor(green); write(' con código '); textcolor(yellow);
writeln(flor.codigo);
end;
writeln();
end;
var
    archivo_flores: t_archivo_flores;
    registro_flor: t_registro_flor;
begin
    randomize;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO FLORES:'); writeln();
    assign(archivo_flores,'E5_flores');
    cargar_archivo_flores(archivo_flores);
    imprimir_archivo_flores(archivo_flores);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO FLORES AL AGREGAR FLOR ANTES DE LA
BAJA:'); writeln();
    registro_flor.codigo:=1+random(100);
    registro_flor.nombre:=random_string(5+random(6));
    agregar_flor(archivo_flores,registro_flor.nombre,registro_flor.codigo);
    imprimir_archivo_flores(archivo_flores);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO FLORES AL ELIMINAR FLOR:'); writeln();
    registro_flor.codigo:=1+random(100);
    eliminar_flor(archivo_flores,registro_flor);
    imprimir_archivo_flores(archivo_flores);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO FLORES AL AGREGAR FLOR DESPUÉS DE LA
BAJA:'); writeln();
    registro_flor.codigo:=1+random(100);
    registro_flor.nombre:=random_string(5+random(6));
    agregar_flor(archivo_flores,registro_flor.nombre,registro_flor.codigo);
    imprimir_archivo_flores(archivo_flores);
end.
```

Ejercicio 6.

Una cadena de tiendas de indumentaria posee un archivo maestro no ordenado con la información correspondiente a las prendas que se encuentran a la venta. De cada prenda, se registra: *cod_prenda*, *descripción*, *colores*, *tipo_prenda*, *stock* y *precio_unitario*. Ante un eventual cambio de temporada, se deben actualizar las prendas a la venta. Para ello, reciben un archivo conteniendo: *cod_prenda* de las prendas que quedarán obsoletas. Se deberá implementar un procedimiento que reciba ambos archivos y realice la baja lógica de las prendas. Para ello, se deberá modificar el *stock* de la prenda correspondiente a valor negativo.

Adicionalmente, se deberá implementar otro procedimiento que se encargue de efectivizar las bajas lógicas que se realizaron sobre el archivo maestro con la información de las prendas a la venta. Para ello, se deberá utilizar una estructura auxiliar (esto es, un archivo nuevo), en el cual se copien, únicamente, aquellas prendas que no están marcadas como borradas. Al finalizar este proceso de compactación del archivo, se deberá renombrar el archivo nuevo con el nombre del archivo maestro original.

```
program TP3_E6;
{$codepage UTF8}
uses crt;
const
  codigo_salida=-1;
  stock_baja=-1;
type
  t_string20=string[20];
  t_registro_prenda=record
    codigo: int16;
    descripcion: t_string20;
    colores: t_string20;
    tipo: t_string20;
    stock: int16;
    precio: real;
  end;
  t_archivo_maestro=file of t_registro_prenda;
  t_archivo_detalle=file of int16;
function random_string(length: int8): t_string20;
var
  i: int8;
  string_aux: string;
begin
  string_aux:='';
  for i:= 1 to length do
    string_aux:=string_aux+chr(ord('A')+random(26));
  random_string:=string_aux;
end;
procedure leer_prenda(var registro_prenda: t_registro_prenda);
var
  i: int8;
begin
  i:=random(100);
  if (i=0) then
    registro_prenda.codigo:=codigo_salida
  else
    registro_prenda.codigo:=1+random(1000);
  if (registro_prenda.codigo<>codigo_salida) then
    begin
      registro_prenda.descripcion:=random_string(10+random(10));
```

```

registro_prenda.colores:=random_string(5+random(6));
registro_prenda.tipo:=random_string(5+random(6));
registro_prenda.stock:=1+random(100);
registro_prenda.precio:=1+random(100);
end;
end;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_prenda: t_registro_prenda;
begin
    rewrite(archivo_maestro);
    leer_prenda(registro_prenda);
    while (registro_prenda.codigo<>codigo_salida) do
    begin
        write(archivo_maestro,registro_prenda);
        leer_prenda(registro_prenda);
    end;
    close(archivo_maestro);
end;
procedure leer_codigo(var codigo: int16);
var
    i: int8;
begin
    i:=random(100);
    if (i=0) then
        codigo:=codigo_salida
    else
        codigo:=1+random(1000);
    end;
end;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    codigo: int16;
begin
    rewrite(archivo_detalle);
    leer_codigo(codigo);
    while (codigo<>codigo_salida) do
    begin
        write(archivo_detalle,codigo);
        leer_codigo(codigo);
    end;
    close(archivo_detalle);
end;
procedure imprimir_registro_prenda(registro_prenda: t_registro_prenda);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_prenda.codigo);
    textcolor(green); write('; Descripción: '); textcolor(yellow);
write(registro_prenda.descripcion);
    textcolor(green); write('; Colores: '); textcolor(yellow); write(registro_prenda.colores);
    textcolor(green); write('; Tipo: '); textcolor(yellow); write(registro_prenda.tipo);
    textcolor(green); write('; Stock: '); textcolor(yellow); write(registro_prenda.stock);
    textcolor(green); write('; Precio: $'); textcolor(yellow);
writeln(registro_prenda.precio:0:2);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_prenda: t_registro_prenda;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
    begin
        read(archivo_maestro,registro_prenda);
        imprimir_registro_prenda(registro_prenda);
    end;
    textcolor(green); write('Tamaño del archivo maestro: '); textcolor(yellow);
writeln(filesize(archivo_maestro));
close(archivo_maestro);

```

```
end;
procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
  codigo: int16;
begin
  reset(archivo_detalle);
  textcolor(green); write('Códigos: ');
  while (not eof(archivo_detalle)) do
    begin
      read(archivo_detalle,codigo);
      textcolor(yellow); write(codigo,' ');
    end;
  close(archivo_detalle);
  writeln();
end;
procedure bajas_logicas(var archivo_maestro: t_archivo_maestro; var archivo_detalle:
t_archivo_detalle);
var
  registro_prenda: t_registro_prenda;
  codigo: int16;
begin
  reset(archivo_maestro);
  reset(archivo_detalle);
  while (not eof(archivo_detalle)) do
    begin
      read(archivo_detalle,codigo);
      seek(archivo_maestro,0);
      read(archivo_maestro,registro_prenda);
      while ((not eof(archivo_maestro)) and (registro_prenda.codigo<>codigo)) do
        read(archivo_maestro,registro_prenda);
      if (registro_prenda.codigo=codigo) then
        begin
          seek(archivo_maestro,filepos(archivo_maestro)-1);
          registro_prenda.stock:=stock_baja;
          write(archivo_maestro,registro_prenda);
        end;
      end;
    close(archivo_maestro);
    close(archivo_detalle);
  end;
procedure bajas_fisicas(var archivo_maestro: t_archivo_maestro);
var
  registro_prenda: t_registro_prenda;
  archivo_maestro_aux: t_archivo_maestro;
begin
  reset(archivo_maestro);
  assign(archivo_maestro_aux,'E6_prendasMaestroAux'); rewrite(archivo_maestro_aux);
  while (not eof(archivo_maestro)) do
    begin
      read(archivo_maestro,registro_prenda);
      if (registro_prenda.stock>0) then
        write(archivo_maestro_aux,registro_prenda);
      end;
    close(archivo_maestro);
    close(archivo_maestro_aux);
    erase(archivo_maestro);
    rename(archivo_maestro_aux,'E6_prendasMaestro');
  end;
var
  archivo_maestro: t_archivo_maestro;
  archivo_detalle: t_archivo_detalle;
begin
  randomize;
  writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
  assign(archivo_maestro,'E6_prendasMaestro');
  cargar_archivo_maestro(archivo_maestro);
```

```
imprimir_archivo_maestro(archivo_maestro);
writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE:'); writeln();
assign(archivo_detalle,'E6_bajasDetalle');
cargar_archivo_detalle(archivo_detalle);
imprimir_archivo_detalle(archivo_detalle);
writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO CON BAJAS LÓGICAS:');
writeln();
bajas_logicas(archivo_maestro,archivo_detalle);
imprimir_archivo_maestro(archivo_maestro);
writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO CON BAJAS FÍSICAS:');
writeln();
bajas_fisicas(archivo_maestro);
imprimir_archivo_maestro(archivo_maestro);
end.
```

Ejercicio 7.

Se cuenta con un archivo que almacena información sobre especies de aves en vía de extinción. Para ello, se almacena: código, nombre de la especie, familia de ave, descripción y zona geográfica. El archivo no está ordenado por ningún criterio. Realizar un programa que permita borrar especies de aves extintas. Este programa debe disponer de dos procedimientos:

(a) Un procedimiento que, dada una especie de ave (su código), marque la misma como borrada (en caso de querer borrar múltiples especies de aves, se podría invocar este procedimiento repetidamente).

(b) Un procedimiento que compacte el archivo, quitando, definitivamente, las especies de aves marcadas como borradas. Para quitar los registros, se deberá copiar el último registro del archivo en la posición del registro a borrar y, luego, eliminar del archivo el último registro de forma tal de evitar registros duplicados.

(i) Implementar una variante de este procedimiento de compactación del archivo (baja física) donde el archivo se trunque una sola vez.

```
program TP3_E7;
{$codepage UTF8}
uses crt;
const
  codigo_salida=-1;
  codigo_baja=-1;
type
  t_string20=string[20];
  t_registro_ave=record
    codigo: int16;
    nombre: t_string20;
    familia: t_string20;
    descripcion: t_string20;
    zona: t_string20;
  end;
  t_archivo_aves=file of t_registro_ave;
function random_string(length: int8): t_string20;
var
  i: int8;
  string_aux: string;
begin
  string_aux:='';
  for i:= 1 to length do
    string_aux:=string_aux+chr(ord('A')+random(26));
  random_string:=string_aux;
end;
procedure leer_ave(var registro_ave: t_registro_ave);
var
  i: int8;
begin
  i:=random(10);
  if (i=0) then
    registro_ave.codigo:=codigo_salida
  else
    registro_ave.codigo:=1+random(1000);
  if (registro_ave.codigo<>codigo_salida) then
    begin
      registro_ave.nombre:=random_string(5+random(6));
```



```
registro_ave.familia:=random_string(5+random(6));
registro_ave.descripcion:=random_string(10+random(10));
registro_ave.zona:=random_string(5+random(6));
end;
end;
procedure cargar_archivo_aves(var archivo_aves: t_archivo_aves);
var
    registro_ave: t_registro_ave;
begin
    rewrite(archivo_aves);
    leer_ave(registro_ave);
    while (registro_ave.codigo<>codigo_salida) do
    begin
        write(archivo_aves,registro_ave);
        leer_ave(registro_ave);
    end;
    close(archivo_aves);
end;
procedure imprimir_registro_ave(registro_ave: t_registro_ave);
begin
    textcolor(green); write('Nombre: '); textcolor(yellow); write(registro_ave.nombre);
    textcolor(green); write('; Código: '); textcolor(yellow); write(registro_ave.codigo);
    textcolor(green); write('; Familia: '); textcolor(yellow); write(registro_ave.familia);
    textcolor(green); write('; Descripción: '); textcolor(yellow);
write(registro_ave.descripcion);
    textcolor(green); write('; Zona geográfica: '); textcolor(yellow);
writeln(registro_ave.zona);
end;
procedure imprimir_archivo_aves(var archivo_aves: t_archivo_aves);
var
    registro_ave: t_registro_ave;
begin
    reset(archivo_aves);
    while (not eof(archivo_aves)) do
    begin
        read(archivo_aves,registro_ave);
        imprimir_registro_ave(registro_ave);
    end;
    textcolor(green); write('Tamaño del archivo aves: '); textcolor(yellow);
writeln(filesize(archivo_aves));
    close(archivo_aves);
end;
procedure leer_codigo(var codigo: int16);
var
    i: int8;
begin
    i:=random(100);
    if (i=0) then
        codigo:=codigo_salida
    else
        codigo:=1+random(1000);
end;
procedure baja_logica(var archivo_aves: t_archivo_aves; codigo: int16);
var
    registro_ave: t_registro_ave;
    ok: boolean;
begin
    ok:=false;
    reset(archivo_aves);
    while (not eof(archivo_aves)) and (ok=false) do
    begin
        read(archivo_aves,registro_ave);
        if (registro_ave.codigo=codigo) then
        begin
            ok:=true;
            registro_ave.codigo:=codigo_baja;
```

```

        seek(archivo_aves,filepos(archivo_aves)-1);
        write(archivo_aves,registro_ave);
    end;
end;
close(archivo_aves);
if (ok=true) then
begin
    textcolor(green); write('Se ha realizado la baja de la especie de ave con código ');
textcolor(yellow); write(codigo); textcolor(green); writeln(' en el archivo');
end
else
begin
    textcolor(green); write('No se ha encontrado la especie de ave con código ');
textcolor(yellow); write(codigo); textcolor(green); writeln(' en el archivo');
end;
end;
procedure bajas_logicas(var archivo_aves: t_archivo_aves);
var
    codigo: int16;
begin
    leer_codigo(codigo);
    while (codigo<>codigo_salida) do
    begin
        baja_logica(archivo_aves,codigo);
        leer_codigo(codigo);
    end;
    writeln();
end;
procedure bajas_fisicas1(var archivo_aves: t_archivo_aves);
var
    registro_ave, registro_ultima_ave: t_registro_ave;
    pos: int16;
begin
    reset(archivo_aves);
    while (not eof(archivo_aves)) do
    begin
        read(archivo_aves,registro_ave);
        if (registro_ave.codigo=codigo_baja) then
        begin
            pos:=filepos(archivo_aves)-1;
            seek(archivo_aves,filesize(archivo_aves)-1);
            read(archivo_aves,registro_ultima_ave);
            seek(archivo_aves,pos);
            write(archivo_aves,registro_ultima_ave);
            seek(archivo_aves,filesize(archivo_aves)-1);
            truncate(archivo_aves);
            seek(archivo_aves,pos);
        end;
    end;
    close(archivo_aves);
end;
procedure bajas_fisicas2(var archivo_aves: t_archivo_aves);
var
    registro_ave, registro_ultima_ave: t_registro_ave;
    pos_act, pos_ult: int16;
begin
    reset(archivo_aves);
    if (not eof(archivo_aves)) then
    begin
        pos_act:=0; pos_ult:=filesize(archivo_aves)-1;
        while (pos_act<=pos_ult) do
        begin
            seek(archivo_aves,pos_act);
            read(archivo_aves,registro_ave);
            if (registro_ave.codigo=codigo_baja) then
            begin

```

```
        seek(archivo_aves,pos_ult);
        read(archivo_aves,registro_ultima_ave);
        seek(archivo_aves,pos_act);
        write(archivo_aves,registro_ultima_ave);
        pos_ult:=pos_ult-1;
    end
    else
        pos_act:=pos_act+1;
    end;
    seek(archivo_aves,pos_ult+1);
    truncate(archivo_aves);
end;
close(archivo_aves);
end;
var
    archivo_aves: t_archivo_aves;
begin
    randomize;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO AVES:'); writeln();
    assign(archivo_aves,'E7_aves');
    cargar_archivo_aves(archivo_aves);
    imprimir_archivo_aves(archivo_aves);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO AVES CON BAJAS LÓGICAS:'); writeln();
    bajas_logicas(archivo_aves);
    imprimir_archivo_aves(archivo_aves);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO AVES CON BAJAS FÍSICAS:'); writeln();
    bajas_fisicas1(archivo_aves);
    imprimir_archivo_aves(archivo_aves);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO AVES CON BAJAS FÍSICAS:'); writeln();
    bajas_fisicas2(archivo_aves);
    imprimir_archivo_aves(archivo_aves);
end.
```

Ejercicio 8.

Se cuenta con un archivo con información de las diferentes distribuciones de Linux existentes. De cada distribución, se conoce: nombre, año de lanzamiento, número de versión del kernel, cantidad de desarrolladores y descripción. El nombre de las distribuciones no puede repetirse. Este archivo debe ser mantenido realizando bajas lógicas y utilizando la técnica de reutilización de espacio libre llamada lista invertida. Escribir la definición de las estructuras de datos necesarias y los siguientes procedimientos:

(a) *BuscarDistribucion*: Módulo que recibe por parámetro el archivo, un nombre de distribución y devuelve la posición dentro del archivo donde se encuentra el registro correspondiente a la distribución dada, si existe, o devuelve -1, en caso de que no exista.

(b) *AltaDistribucion*: Módulo que recibe como parámetro el archivo y el registro que contiene los datos de una nueva distribución, y se encarga de agregar la distribución al archivo reutilizando espacio disponible en caso de que exista. (El control de unicidad se debe realizar utilizando el módulo anterior). En caso de que la distribución que se quiere agregar ya exista, se debe informar “Ya existe la distribución”.

(c) *BajaDistribucion*: Módulo que recibe como parámetro el archivo y el nombre de una distribución, y se encarga de dar de baja lógicamente la distribución dada. Para marcar una distribución como borrada, se debe utilizar el campo cantidad de desarrolladores para mantener actualizada la lista invertida. Para verificar que la distribución a borrar exista, se debe utilizar el módulo *BuscarDistribucion*. En caso de no existir, se debe informar “Distribución no existente”.

```
program TP3_E8;
{$codepage UTF8}
uses crt;
const
  nombre_salida='ZZZ';
  desarrolladores_cabecera=0;
type
  t_string20=string[20];
  t_registro_distribucion=record
    nombre: t_string20;
    anio: int16;
    kernel: int16;
    desarrolladores: int16;
    descripcion: t_string20;
  end;
  t_archivo_distribuciones=file of t_registro_distribucion;
function random_string(length: int8): t_string20;
var
  i: int8;
  string_aux: string;
begin
  string_aux:='';
  for i:= 1 to length do
    string_aux:=string_aux+chr(ord('A')+random(26));
  end;
  random_string:=string_aux;
end;
procedure leer_cabecera(var registro_distribucion: t_registro_distribucion);
begin
  registro_distribucion.nombre:='Cabecera';
```

```

registro_distribucion.anio:=desarrolladores_cabecera;
registro_distribucion.kernel:=desarrolladores_cabecera;
registro_distribucion.desarrolladores:=desarrolladores_cabecera;
registro_distribucion.descripcion:='Cabecera';
end;
procedure leer_distribucion(var registro_distribucion: t_registro_distribucion);
var
  i: int8;
begin
  i:=random(10);
  if (i=0) then
    registro_distribucion.nombre:=nombre_salida
  else
    registro_distribucion.nombre:=random_string(5+random(6));
    if (registro_distribucion.nombre<>nombre_salida) then
      begin
        registro_distribucion.anio:=2000+random(26);
        registro_distribucion.kernel:=1+random(10);
        registro_distribucion.desarrolladores:=1+random(100);
        registro_distribucion.descripcion:=random_string(10+random(10));
      end;
    end;
end;
procedure cargar_archivo_distribuciones(var archivo_distribuciones: t_archivo_distribuciones);
var
  registro_distribucion: t_registro_distribucion;
begin
  rewrite(archivo_distribuciones);
  leer_cabecera(registro_distribucion);
  while (registro_distribucion.nombre<>nombre_salida) do
    begin
      write(archivo_distribuciones,registro_distribucion);
      leer_distribucion(registro_distribucion);
    end;
  close(archivo_distribuciones);
end;
procedure imprimir_registro_distribucion(registro_distribucion: t_registro_distribucion);
begin
  textcolor(green); write('Nombre: '); textcolor(yellow); write(registro_distribucion.nombre);
  textcolor(green); write('; Año: '); textcolor(yellow); write(registro_distribucion.anio);
  textcolor(green); write('; Kernel: '); textcolor(yellow);
write(registro_distribucion.kernel);
  textcolor(green); write('; Desarrolladores: '); textcolor(yellow);
write(registro_distribucion.desarrolladores);
  textcolor(green); write('; Descripción: '); textcolor(yellow);
writeln(registro_distribucion.descripcion);
end;
procedure imprimir_archivo_distribuciones(var archivo_distribuciones:
t_archivo_distribuciones);
var
  registro_distribucion: t_registro_distribucion;
begin
  reset(archivo_distribuciones);
  while (not eof(archivo_distribuciones)) do
    begin
      read(archivo_distribuciones,registro_distribucion);
      imprimir_registro_distribucion(registro_distribucion);
    end;
    textcolor(green); write('Tamaño del archivo distribuciones: '); textcolor(yellow);
writeln(filesize(archivo_distribuciones));
  close(archivo_distribuciones);
end;
procedure buscar_distribucion(var archivo_distribuciones: t_archivo_distribuciones; nombre:
t_string20; var pos: int16);
var
  registro_distribucion: t_registro_distribucion;
  ok: boolean;

```

```

begin
    ok:=false;
    reset(archivo_distribuciones);
    while (not eof(archivo_distribuciones)) and (ok=false) do
    begin
        read(archivo_distribuciones,registro_distribucion);
        if (registro_distribucion.nombre=nombre) then
            ok:=true;
        end;
        if (ok=true) then
            pos:=filepos(archivo_distribuciones)-1
        else
            pos:=-1;
        close(archivo_distribuciones);
    end;
procedure alta_distribucion(var archivo_distribuciones: t_archivo_distribuciones;
registro_distribucion: t_registro_distribucion);
var
    cabecera: t_registro_distribucion;
    pos: int16;
begin
    buscar_distribucion(archivo_distribuciones,registro_distribucion.nombre,pos);
    if (pos=-1) then
    begin
        reset(archivo_distribuciones);
        read(archivo_distribuciones,cabecera);
        if (cabecera.desarrolladores=desarrolladores_cabecera) then
        begin
            seek(archivo_distribuciones,filesize(archivo_distribuciones));
            write(archivo_distribuciones,registro_distribucion);
        end
        else
        begin
            seek(archivo_distribuciones,cabecera.desarrolladores*(-1));
            read(archivo_distribuciones,cabecera);
            seek(archivo_distribuciones,filepos(archivo_distribuciones)-1);
            write(archivo_distribuciones,registro_distribucion);
            seek(archivo_distribuciones,0);
            write(archivo_distribuciones,cabecera);
        end;
        close(archivo_distribuciones);
        textcolor(green); write('Se ha realizado el alta de la distribución '); textcolor(yellow);
write(registro_distribucion.nombre); textcolor(green); writeln(' en el archivo');
    end
    else
    begin
        textcolor(green); write('Ya existe la distribución '); textcolor(yellow);
write(registro_distribucion.nombre); textcolor(green); writeln(' en el archivo');
    end;
    writeln();
end;
procedure baja_distribucion(var archivo_distribuciones: t_archivo_distribuciones; nombre:
t_string20);
var
    registro_distribucion, cabecera: t_registro_distribucion;
    pos: int16;
begin
    buscar_distribucion(archivo_distribuciones,nombre,pos);
    if (pos<>-1) then
    begin
        reset(archivo_distribuciones);
        read(archivo_distribuciones,cabecera);
        read(archivo_distribuciones,registro_distribucion);
        while (registro_distribucion.nombre<>nombre) do
            read(archivo_distribuciones,registro_distribucion);
        seek(archivo_distribuciones,filepos(archivo_distribuciones)-1);
    end;
end;

```

```

write(archivo_distribuciones,cabecera);
cabecera.desarrolladores:=(filepos(archivo_distribuciones)-1)*(-1);
seek(archivo_distribuciones,0);
write(archivo_distribuciones,cabecera);
close(archivo_distribuciones);
textcolor(green); write('Se ha realizado la baja de la distribución '); textcolor(yellow);
write(nombre); textcolor(green); writeln(' en el archivo');
end
else
begin
textcolor(green); write('No se ha encontrado la distribución '); textcolor(yellow);
write(nombre); textcolor(green); writeln(' en el archivo');
end;
writeln();
end;
var
archivo_distribuciones: t_archivo_distribuciones;
registro_distribucion: t_registro_distribucion;
pos: int16;
begin
randomize;
writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DISTRIBUCIONES:'); writeln();
assign(archivo_distribuciones,'E8_distribuciones');
cargar_archivo_distribuciones(archivo_distribuciones);
imprimir_archivo_distribuciones(archivo_distribuciones);
writeln(); textcolor(red); writeln('MÓDULO BuscarDistribucion:'); writeln();
registro_distribucion.nombre:=random_string(5+random(6));
buscar_distribucion(archivo_distribuciones,registro_distribucion.nombre,pos);
if (pos<>-1) then
begin
textcolor(green); write('Se encontró la distribución '); textcolor(yellow);
write(registro_distribucion.nombre); textcolor(green); write(' en la posición ');
textcolor(red); write(pos); textcolor(green); writeln(' del archivo');
end
else
begin
textcolor(green); write('No se encontró la distribución '); textcolor(yellow);
write(registro_distribucion.nombre); textcolor(green); writeln(' en el archivo');
end;
writeln(); textcolor(red); writeln('MÓDULO AltaDistribucion:'); writeln();
leer_distribucion(registro_distribucion);
while (registro_distribucion.nombre=nombre_salida) do
leer_distribucion(registro_distribucion);
alta_distribucion(archivo_distribuciones,registro_distribucion);
imprimir_archivo_distribuciones(archivo_distribuciones);
writeln(); textcolor(red); writeln('MÓDULO BajaDistribucion:'); writeln();
registro_distribucion.nombre:=random_string(5+random(6));
baja_distribucion(archivo_distribuciones,registro_distribucion.nombre);
imprimir_archivo_distribuciones(archivo_distribuciones);
end.

```

PARTE II: Actualización Maestro/Detalle, Reportes y Merge con Archivos no Ordenados.

Para los ejercicios de esta parte de la práctica, teniendo en cuenta que los archivos no están ordenados por ningún criterio, puede resultar necesario recorrer los archivos más de una vez. La idea es resolver los ejercicios sin ordenar los archivos dados y comparar la eficiencia (en cuanto al número de lecturas/escrituras) de la solución brindada en esta práctica respecto a la solución para el mismo problema considerando los archivos ordenados.

Ejercicio 9.

El encargado de ventas de un negocio de productos de limpieza desea administrar el stock de los productos que vende. Para ello, genera un archivo maestro donde figuran todos los productos que comercializa. De cada producto, se maneja la siguiente información: código de producto, nombre comercial, precio de venta, stock actual y stock mínimo. Diariamente, se genera un archivo detalle donde se registran todas las ventas de productos realizadas. De cada venta, se registran: código de producto y cantidad de unidades vendidas. Resolver los siguientes puntos:

(a) Se pide realizar un procedimiento que actualice el archivo maestro con el archivo detalle, teniendo en cuenta que:

- Los archivos no están ordenados por ningún criterio.
- Cada registro del maestro puede ser actualizado por 0, 1 o más registros del archivo detalle.

(b) ¿Qué cambios se realizarían en el procedimiento del inciso anterior si se sabe que cada registro del archivo maestro puede ser actualizado por 0 o 1 registro del archivo detalle?

```
program TP3_E9;
{$codepage UTF8}
uses crt;
const
  codigo_salida=-1;
type
  t_string10=string[10];
  t_registro_producto=record
    codigo: int16;
    nombre: t_string10;
    precio: real;
    stock_actual: int16;
    stock_minimo: int16;
  end;
  t_registro_venta=record
    codigo: int16;
    cantidad_vendida: int16;
  end;
  t_archivo_maestro=file of t_registro_producto;
  t_archivo_detalle=file of t_registro_venta;
function random_string(length: int8): t_string10;
var
  i: int8;
```



```

    string_aux: string;
begin
    string_aux:='';
    for i:= 1 to length do
        string_aux:=string_aux+chr(ord('A')+random(26));
        random_string:=string_aux;
    end;
procedure leer_producto(var registro_producto: t_registro_producto);
var
    i: int8;
begin
    i:=random(10);
    if (i=0) then
        registro_producto.codigo:=codigo_salida
    else
        registro_producto.codigo:=1+random(1000);
        if (registro_producto.codigo<>codigo_salida) then
            begin
                registro_producto.nombre:=random_string(5+random(5));
                registro_producto.precio:=1+random(100);
                registro_producto.stock_actual:=10+random(91);
                registro_producto.stock_minimo:=1+random(50);
            end;
        end;
end;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_producto: t_registro_producto;
begin
    rewrite(archivo_maestro);
    leer_producto(registro_producto);
    while (registro_producto.codigo<>codigo_salida) do
        begin
            write(archivo_maestro,registro_producto);
            leer_producto(registro_producto);
        end;
    close(archivo_maestro);
end;
procedure leer_venta(var registro_venta: t_registro_venta);
var
    i: int8;
begin
    i:=random(100);
    if (i=0) then
        registro_venta.codigo:=codigo_salida
    else
        registro_venta.codigo:=1+random(1000);
        if (registro_venta.codigo<>codigo_salida) then
            registro_venta.cantidad_vendida:=1+random(10);
        end;
end;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    registro_venta: t_registro_venta;
begin
    rewrite(archivo_detalle);
    leer_venta(registro_venta);
    while (registro_venta.codigo<>codigo_salida) do
        begin
            write(archivo_detalle,registro_venta);
            leer_venta(registro_venta);
        end;
    close(archivo_detalle);
end;
procedure imprimir_registro_producto(registro_producto: t_registro_producto);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_producto.codigo);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_producto.nombre);

```

```

    textcolor(green); write('; Precio: $'); textcolor(yellow);
write(registro_producto.precio:0:2);
    textcolor(green); write('; Stock actual: '); textcolor(yellow);
write(registro_producto.stock_actual);
    textcolor(green); write('; Stock mínimo: '); textcolor(yellow);
writeln(registro_producto.stock_minimo);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_producto: t_registro_producto;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro,registro_producto);
            imprimir_registro_producto(registro_producto);
        end;
        textcolor(green); write('Tamaño del archivo maestro: '); textcolor(yellow);
writeln(filesize(archivo_maestro));
        close(archivo_maestro);
    end;
procedure imprimir_registro_venta(registro_venta: t_registro_venta);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_venta.codigo);
    textcolor(green); write('; Cantidad vendida: '); textcolor(yellow);
writeln(registro_venta.cantidad_vendida);
end;
procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    registro_venta: t_registro_venta;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
        begin
            read(archivo_detalle,registro_venta);
            imprimir_registro_venta(registro_venta);
        end;
        close(archivo_detalle);
    end;
procedure actualizar1_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_detalle: t_archivo_detalle);
var
    registro_producto: t_registro_producto;
    registro_venta: t_registro_venta;
    ventas: int16;
begin
    reset(archivo_maestro);
    reset(archivo_detalle);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro,registro_producto);
            ventas:=0;
            seek(archivo_detalle,0);
            while (not eof(archivo_detalle)) do
                begin
                    read(archivo_detalle,registro_venta);
                    if (registro_venta.codigo=registro_producto.codigo) then
                        ventas:=ventas+registro_venta.cantidad_vendida;
                    end;
                if (ventas>0) then
                    begin
                        registro_producto.stock_actual:=registro_producto.stock_actual-ventas;
                        seek(archivo_maestro,filepos(archivo_maestro)-1);
                        write(archivo_maestro,registro_producto);
                    end;
                end;
            end;
        end;
end;

```

```

    close(archivo_maestro);
    close(archivo_detalle);
end;
procedure actualizar2_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_detalle: t_archivo_detalle);
var
    registro_producto: t_registro_producto;
    registro_venta: t_registro_venta;
begin
    reset(archivo_maestro);
    reset(archivo_detalle);
    while (not eof(archivo_maestro)) do
    begin
        read(archivo_maestro,registro_producto);
        seek(archivo_detalle,0);
        read(archivo_detalle,registro_venta);
        while ((not eof(archivo_detalle)) and (registro_venta.codigo<>registro_producto.codigo))
do
            read(archivo_detalle,registro_venta);
            if (registro_venta.codigo=registro_producto.codigo) then
            begin
                registro_producto.stock_actual:=registro_producto.stock_actual-
registro_venta.cantidad_vendida;
                seek(archivo_maestro,filepos(archivo_maestro)-1);
                write(archivo_maestro,registro_producto);
            end;
        end;
        close(archivo_maestro);
        close(archivo_detalle);
    end;
var
    archivo_maestro: t_archivo_maestro;
    archivo_detalle: t_archivo_detalle;
begin
    randomize;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E9_productosMaestro');
    cargar_archivo_maestro(archivo_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE:'); writeln();
    assign(archivo_detalle,'E9_productosDetalle');
    cargar_archivo_detalle(archivo_detalle);
    imprimir_archivo_detalle(archivo_detalle);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO 1:'); writeln();
    actualizar1_archivo_maestro(archivo_maestro,archivo_detalle);
    imprimir_archivo_maestro(archivo_maestro);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO 2:'); writeln();
    actualizar2_archivo_maestro(archivo_maestro,archivo_detalle);
    imprimir_archivo_maestro(archivo_maestro);
end.

```

Ejercicio 10.

Se necesita contabilizar los votos de las diferentes mesas electorales registradas por localidad en la Provincia de Buenos Aires. Para ello, se posee un archivo con la siguiente información: código de localidad, número de mesa y cantidad de votos en dicha mesa. Presentar en pantalla un listado como se muestra a continuación:

Código de Localidad	Total de Votos
.....	
.....	
Total General de Votos:	

Notas:

- La información en el archivo no está ordenada por ningún criterio.
- Tratar de resolver el problema sin modificar el contenido del archivo dado.
- Se puede utilizar una estructura auxiliar, como por ejemplo otro archivo, para llevar el control de las localidades que han sido procesadas.

```

program TP3_E10;
{$codepage UTF8}
uses crt;
const
    localidad_salida=-1;
type
    t_registro_mesa=record
        localidad: int16;
        numero: int16;
        votos: int32;
    end;
    t_archivo_mesas=file of t_registro_mesa;
procedure leer_mesa(var registro_mesa: t_registro_mesa);
var
    i: int8;
begin
    i:=random(10);
    if (i=0) then
        registro_mesa.localidad:=localidad_salida
    else
        registro_mesa.localidad:=1+random(100);
        if (registro_mesa.localidad<>localidad_salida) then
            begin
                registro_mesa.numero:=1+random(100);
                registro_mesa.votos:=10+random(991);
            end;
        end;
end;
procedure cargar_archivo_mesas(var archivo_mesas: t_archivo_mesas);
var
    registro_mesa: t_registro_mesa;
begin
    rewrite(archivo_mesas);
    leer_mesa(registro_mesa);
    while (registro_mesa.localidad<>localidad_salida) do

```

```

begin
    write(archivo_mesas, registro_mesa);
    leer_mesa(registro_mesa);
end;
close(archivo_mesas);
end;
procedure imprimir_registro_mesa(registro_mesa: t_registro_mesa);
begin
    textcolor(green); write('Código de localidad: '); textcolor(yellow);
write(registro_mesa.localidad);
    textcolor(green); write('; Número de mesa: '); textcolor(yellow);
write(registro_mesa.numero);
    textcolor(green); write('; Votos: '); textcolor(yellow); writeln(registro_mesa.votos);
end;
procedure imprimir_archivo_mesas(var archivo_mesas: t_archivo_mesas);
var
    registro_mesa: t_registro_mesa;
begin
    reset(archivo_mesas);
    while (not eof(archivo_mesas)) do
        begin
            read(archivo_mesas, registro_mesa);
            imprimir_registro_mesa(registro_mesa);
        end;
        textcolor(green); write('Tamaño del archivo mesas: '); textcolor(yellow);
writeln(filesize(archivo_mesas));
        close(archivo_mesas);
    end;
procedure cargar_archivo_mesas_aux(var archivo_mesas: t_archivo_mesas; var archivo_mesas_aux:
t_archivo_mesas);
var
    registro_mesa, registro_mesa_aux: t_registro_mesa;
    ok: boolean;
begin
    reset(archivo_mesas);
    rewrite(archivo_mesas_aux);
    while (not eof(archivo_mesas)) do
        begin
            ok:=false;
            read(archivo_mesas, registro_mesa);
            seek(archivo_mesas_aux, 0);
            while (not eof(archivo_mesas_aux)) and (ok=false) do
                begin
                    read(archivo_mesas_aux, registro_mesa_aux);
                    if (registro_mesa_aux.localidad=registro_mesa.localidad) then
                        ok:=true;
                    end;
                    if (ok=true) then
                        begin
                            registro_mesa_aux.votos:=registro_mesa_aux.votos+registro_mesa.votos;
                            seek(archivo_mesas_aux, filepos(archivo_mesas_aux)-1);
                            write(archivo_mesas_aux, registro_mesa_aux);
                        end
                    else
                        write(archivo_mesas_aux, registro_mesa);
                    end;
                end;
            close(archivo_mesas);
            close(archivo_mesas_aux);
        end;
procedure imprimir_archivo_mesas_aux(var archivo_mesas: t_archivo_mesas);
var
    registro_mesa: t_registro_mesa;
    votos_total: int32;
begin
    votos_total:=0;
    reset(archivo_mesas);

```

```
textcolor(green); writeln('Código de Localidad      Total de Votos');
while (not eof(archivo_mesas)) do
begin
    read(archivo_mesas,registro_mesa);
    textcolor(yellow); write(registro_mesa.localidad); textcolor(green);
write('      '); textcolor(red); writeln(registro_mesa.votos);
    votos_total:=votos_total+registro_mesa.votos;
end;
textcolor(green); write('Total General de Votos:      '); textcolor(red);
writeln(votos_total);
close(archivo_mesas);
end;
var
    archivo_mesas, archivo_mesas_aux: t_archivo_mesas;
begin
    randomize;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MESAS:'); writeln();
    assign(archivo_mesas,'E10_mesas');
    cargar_archivo_mesas(archivo_mesas);
    imprimir_archivo_mesas(archivo_mesas);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MESAS AUXILIAR:'); writeln();
    assign(archivo_mesas_aux,'E10_mesas_aux');
    cargar_archivo_mesas_aux(archivo_mesas,archivo_mesas_aux);
    imprimir_archivo_mesas(archivo_mesas_aux);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MESAS AUXILIAR:'); writeln();
    imprimir_archivo_mesas_aux(archivo_mesas_aux);
end.
```

Ejercicio 11.

Suponer que se trabaja en una oficina donde está montada una LAN (red local). La misma fue construida sobre una topología de red que conecta 5 máquinas entre sí y todas las máquinas se conectan con un servidor central. Semanalmente, cada máquina genera un archivo de logs informando las sesiones abiertas por cada usuario en cada terminal y por cuánto tiempo estuvo abierta. Cada archivo detalle contiene los siguientes campos: *cod_usuario*, *fecha*, *tiempo_sesion*. Se debe realizar un procedimiento que reciba los archivos detalle y genere un archivo maestro con los siguientes datos: *cod_usuario*, *fecha*, *tiempo_total_de_sesiones_abiertas*.

Notas:

- Los archivos detalle no están ordenados por ningún criterio.
- Un usuario puede iniciar más de una sesión el mismo día en la misma máquina o, inclusive, en diferentes máquinas.

```
program TP3_E11;
{$codepage UTF8}
uses crt, sysutils;
const
    detalles_total=5;
    codigo_salida=-1;
type
    t_detalle=1..detalles_total;
    t_string20=string[20];
    t_registro_sesion=record
        codigo: int16;
        fecha: t_string20;
        tiempo: int16;
    end;
    t_archivo_sesiones=file of t_registro_sesion;
    t_vector_detalle=array[t_detalle] of t_archivo_sesiones;
procedure leer_sesion(var registro_sesion: t_registro_sesion);
var
    i: int8;
begin
    i:=random(10);
    if (i=0) then
        registro_sesion.codigo:=codigo_salida
    else
        registro_sesion.codigo:=1+random(10);
        if (registro_sesion.codigo<>codigo_salida) then
            begin
                registro_sesion.fecha:='2025-01-'+inttoStr(1+random(30));
                registro_sesion.tiempo:=1+random(100);
            end;
        end;
end;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_sesiones);
var
    registro_sesion: t_registro_sesion;
begin
    rewrite(archivo_detalle);
    leer_sesion(registro_sesion);
    while (registro_sesion.codigo<>codigo_salida) do
        begin
            write(archivo_detalle,registro_sesion);
            leer_sesion(registro_sesion);
        end;
end;
```

```

    close(archivo_detalle);
end;
procedure imprimir_registro_sesion(registro_sesion: t_registro_sesion);
begin
    textcolor(green); write('Código de usuario: '); textcolor(yellow);
write(registro_sesion.codigo);
    textcolor(green); write('; Fecha: '); textcolor(yellow); write(registro_sesion.fecha);
    textcolor(green); write('; Tiempo: '); textcolor(yellow); writeln(registro_sesion.tiempo);
end;
procedure imprimir_archivo_sesiones(var archivo_sesiones: t_archivo_sesiones);
var
    registro_sesion: t_registro_sesion;
begin
    reset(archivo_sesiones);
    while (not eof(archivo_sesiones)) do
        begin
            read(archivo_sesiones, registro_sesion);
            imprimir_registro_sesion(registro_sesion);
        end;
        textcolor(green); write('Tamaño del archivo sesiones: '); textcolor(yellow);
writeln(filesize(archivo_sesiones));
        close(archivo_sesiones);
    end;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_sesiones; var vector_detalles:
t_vector_detalles);
var
    registro_sesion_detalle, registro_sesion_maestro: t_registro_sesion;
    i: t_detalle;
    ok: boolean;
begin
    rewrite(archivo_maestro);
    for i:= 1 to detalles_total do
        begin
            reset(vector_detalles[i]);
            while (not eof(vector_detalles[i])) do
                begin
                    ok:=false;
                    read(vector_detalles[i], registro_sesion_detalle);
                    seek(archivo_maestro, 0);
                    while (not eof(archivo_maestro)) and (ok=false) do
                        begin
                            read(archivo_maestro, registro_sesion_maestro);
                            if ((registro_sesion_maestro.codigo=registro_sesion_detalle.codigo) and
(registro_sesion_maestro.fecha=registro_sesion_detalle.fecha)) then
                                ok:=true;
                            end;
                            if (ok=true) then
                                begin
                                    registro_sesion_maestro.tiempo:=registro_sesion_maestro.tiempo+registro_sesion_detalle
.tiempo;
                                    seek(archivo_maestro, filepos(archivo_maestro)-1);
                                    write(archivo_maestro, registro_sesion_maestro);
                                end
                            else
                                write(archivo_maestro, registro_sesion_detalle);
                            end;
                            close(vector_detalles[i]);
                        end;
                    close(archivo_maestro);
                end;
            end;
        var
            vector_detalles: t_vector_detalles;
            archivo_maestro: t_archivo_sesiones;
            i: t_detalle;
        begin
            randomize;

```



```
for i:= 1 to detalles_total do
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE ',i,':'); writeln();
    assign(vector_detalle[i], 'E11_sesionesDetalle'+inttoStr(i));
    cargar_archivo_detalle(vector_detalle[i]);
    imprimir_archivo_sesiones(vector_detalle[i]);
end;
writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
assign(archivo_maestro, 'E11_sesionesMaestro');
cargar_archivo_maestro(archivo_maestro, vector_detalle);
imprimir_archivo_sesiones(archivo_maestro);
end.
```